

GRAVEATLAS

Phase 7 — Reliability, Observability, Backups, CI/CD & Production Operations

Full copy-paste implementation prompt for the Base44 agent.

COPY-PASTE MASTER PROMPT

GRAVEATLAS — PHASE 7

RELIABILITY, OBSERVABILITY, BACKUPS, CI/CD & PRODUCTION OPERATIONS

PHASE 1 THROUGH PHASE 6 MUST ALREADY BE COMPLETE.

Continue from the existing GraveAtlas implementation.

IMPORTANT:

- Do NOT rebuild Phases 1–6.
- Reuse the existing Android app, Cloudflare Worker/API, GitHub App, public data repository, schemas, search, map, authentication, etc.
- Do NOT start Phase 8.
- Do NOT add paid services without explicit approval.
- Prefer existing/free infrastructure.
- Do NOT expose secrets, tokens, private keys or private user data.
- Do NOT fabricate monitoring results, backups, tests, uptime or recovery results.
- Do NOT perform destructive production operations.
- Do NOT delete legitimate cemetery or historical data.
- Do NOT silently change production architecture.
- Complete only Phase 7, then stop.

=====

PART 1 — PRODUCTION ARCHITECTURE REVIEW

=====

Review the complete production architecture.

Document actual components:

- Android application
- backend/API
- Cloudflare services
- GitHub App
- GitHub repository
- public data
- authentication
- database/storage
- cache
- queues/jobs
- CI/CD
- monitoring/logging

Identify dependencies and failure points.

=====

PART 2 — ENVIRONMENT SEPARATION

=====

Verify separation between:

DEVELOPMENT
STAGING
PRODUCTION

If staging does not exist, document that fact rather than pretending it does.

Prevent accidental production writes from development/testing workflows.

=====

PART 3 — CONFIGURATION MANAGEMENT

=====

Review configuration.

Separate:

- application configuration
- environment configuration
- secrets

- feature flags
- public constants

Do not hard-code production secrets.

=====

PART 4 – HEALTH CHECKS

=====

Implement or verify safe health endpoints.

At minimum distinguish:

- application reachable
- backend reachable
- critical dependency availability

Do not expose secret configuration through health responses.

=====

PART 5 – READINESS CHECKS

=====

Where appropriate, implement readiness checks.

A service should not be considered ready if required dependencies are unavailable.

Avoid making optional dependencies block the entire service unnecessarily.

=====

PART 6 – LIVENESS

=====

Verify that liveness checks detect an unhealthy process without causing unnecessary restarts.

Do not create health-check loops.

=====

PART 7 – ERROR MONITORING

=====

Establish safe error monitoring using existing infrastructure where possible.

Track categories such as:

- API failures
- authentication failures
- publication failures
- GitHub API failures
- data validation failures
- Android crashes if existing tooling supports it

Do not collect unnecessary personal information.

=====

PART 8 – STRUCTURED LOGGING

=====

Review backend logs.

Use structured fields where appropriate:

- timestamp
- request/correlation ID
- operation
- status
- duration
- safe error category

Never log:

- passwords
- tokens
- private keys
- authentication headers
- private moderation notes
- unnecessary precise location

=====

PART 9 – CORRELATION IDS

=====

Where practical, use a request/correlation ID across:

Android
→ API
→ backend operation
→ GitHub/publication

This should help diagnose failures without exposing sensitive data.

=====

PART 10 – METRICS

=====

Define useful operational metrics.

Examples:

- request count
- error rate
- latency
- search latency
- publication success/failure
- queue depth
- retry count
- authentication failures

Do not add unnecessary paid monitoring.

=====

PART 11 – ALERT THRESHOLDS

=====

Define practical alert conditions.

Examples:

- elevated API error rate
- repeated publication failures
- queue stuck
- GitHub authentication failure
- excessive authentication failures
- unusual request volume

Document actual thresholds.

=====

PART 12 – BACKUP INVENTORY

=====

Identify what requires backup:

- application data
- contribution data
- moderation/audit data
- configuration
- public dataset
- infrastructure configuration

Do not assume GitHub alone is a complete backup for private application data.

=====

PART 13 – BACKUP POLICY

=====

Define:

- backup frequency
- retention
- encryption
- access control
- storage location
- restoration process

If a capability is not implemented, explicitly state:

NOT IMPLEMENTED

=====

PART 14 – PUBLIC DATA RECOVERY

=====

Verify that the public dataset can be reconstructed from:

- Git history
- approved source data
- publication records
- validated releases

Do not claim recoverability without testing or evidence.

=====

PART 15 – PRIVATE DATA RECOVERY

=====

Verify recovery for:

- contributions
- moderation
- audit records
- account data where applicable

Ensure recovery does not expose private data.

=====

PART 16 – RESTORE TEST

=====

Perform a safe restoration test in a non-production environment where possible.

Record:

- what was restored
- source
- duration
- result
- missing components
- remediation

Do NOT claim a successful restore without actually testing it.

=====

PART 17 – DISASTER RECOVERY

=====

Define recovery procedures for:

- API outage
- GitHub outage
- Cloudflare outage
- corrupted public data
- failed deployment
- compromised credential
- accidental configuration change

=====

PART 18 – RPO / RTO

=====

Define realistic targets for:

RPO – Recovery Point Objective
RTO – Recovery Time Objective

Do not invent guarantees.

Document assumptions and limitations.

=====

PART 19 – CI PIPELINE REVIEW

=====

Review GitHub Actions / CI.

Verify pipeline stages where applicable:

- checkout
- dependency installation
- validation

- tests
- build
- security checks
- artifact generation

Avoid unnecessary external services.

=====

PART 20 – ANDROID BUILD

=====

Verify reproducible Android builds.

Check:

- debug vs release
- signing configuration
- versioning
- build artifacts
- dependency resolution

Never expose signing keys.

=====

PART 21 – RELEASE ARTIFACTS

=====

Define safe handling for:

- APK
- AAB
- checksums
- build metadata
- release notes

Do not publish test/debug artifacts as production releases.

=====

PART 22 – BACKEND DEPLOYMENT

=====

Review backend deployment.

Verify:

- configuration
- environment
- migrations
- rollback
- health checks
- post-deployment validation

Do not deploy untested changes automatically to production unless the existing policy explicitly permits it.

=====

PART 23 – ROLLBACK

=====

Create a safe rollback strategy.

Support rollback for:

- Android release
- backend deployment
- public data publication
- configuration

Public historical data must not be destructively reverted merely because an application release is rolled back.

=====

PART 24 – DATABASE / DATA MIGRATIONS

=====

Review migrations.

Every migration must be:

- versioned
- testable
- reversible where practical

- documented

Never run destructive migration automatically without explicit authorization.

=====

PART 25 – DEPENDENCY UPDATES

=====

Create a controlled dependency update process.

Separate:

- security update
- minor update
- major update

Test before production deployment.

Do not blindly upgrade all dependencies.

=====

PART 26 – SCHEDULED JOBS

=====

Review scheduled jobs.

Each job must have:

- unique identity
- schedule
- timeout
- retry policy
- failure handling
- idempotency where required

Prevent overlapping executions when unsafe.

=====

PART 27 – JOB RECOVERY

=====

Verify recovery from:

- timeout
- worker crash
- API outage
- GitHub outage
- partial completion

Do not lose job state.

=====

PART 28 – QUEUE MONITORING

=====

If queues exist, monitor:

- pending
- processing
- successful
- failed
- retrying
- stuck

Document how stuck jobs are recovered.

=====

PART 29 – RATE / RESOURCE LIMITS

=====

Review:

- API limits
- GitHub limits
- Cloudflare limits
- Android memory
- request size
- file size
- queue capacity

Avoid unbounded resource consumption.

=====

PART 30 – COST CONTROL

=====

Review infrastructure for unnecessary paid usage.

Prefer:

- existing services
- free tiers
- local processing
- GitHub Actions where appropriate
- Cloudflare existing capabilities

Do not add paid infrastructure without explicit approval.

=====

PART 31 – PRODUCTION CHECKLIST

=====

Create a repeatable pre-release checklist covering:

- tests
- security
- data validation
- migrations
- secrets
- build
- deployment
- health
- rollback
- monitoring

=====

PART 32 – POST-DEPLOYMENT CHECK

=====

After a deployment, verify actual:

- health
- API response
- authentication
- public search
- map
- contributions
- moderation
- publication
- Android compatibility where applicable

Do not claim success without actual checks.

=====

PART 33 – INCIDENT OPERATIONS

=====

Review operational procedures for:

- outage
- security incident
- data corruption
- failed publication
- failed deployment
- GitHub outage
- excessive traffic

Link to Phase 6 security incident procedures.

=====

PART 34 – RUNBOOKS

=====

Create/update:

docs/OPERATIONS.md
docs/HEALTH-CHECKS.md
docs/MONITORING.md
docs/BACKUP-RECOVERY.md
docs/DISASTER-RECOVERY.md

docs/CI-CD.md
docs/DEPLOYMENT.md
docs/ROLLBACK.md
docs/RUNBOOKS.md

Document actual procedures only.

=====

PART 35 — OPERATIONAL TESTING

=====

Test safely:

- health checks
- failure handling
- API outage behavior
- GitHub outage behavior
- publication retry
- scheduled job recovery
- queue recovery
- backup restoration
- rollback procedure
- CI failure
- Android build

Do not perform destructive production tests.

=====

PHASE 7 ACCEPTANCE GATE

=====

Verify:

- [] Production architecture
- [] Environment separation
- [] Configuration management
- [] Health checks
- [] Readiness
- [] Liveness
- [] Error monitoring
- [] Structured logging
- [] Correlation IDs
- [] Metrics
- [] Alerts
- [] Backup inventory
- [] Backup policy
- [] Public data recovery
- [] Private data recovery
- [] Restore test
- [] Disaster recovery
- [] RPO/RTO
- [] CI pipeline
- [] Android build
- [] Release artifacts
- [] Backend deployment
- [] Rollback
- [] Data migrations
- [] Dependency updates
- [] Scheduled jobs
- [] Job recovery
- [] Queue monitoring
- [] Resource limits
- [] Cost control
- [] Production checklist
- [] Post-deployment checks
- [] Incident operations
- [] Runbooks
- [] Operational tests
- [] Documentation

=====

FINAL REPORT

=====

Return:

PHASE 7 STATUS:
READY / NOT READY

PRODUCTION ARCHITECTURE:

PASS / FAIL

HEALTH:
PASS / FAIL

MONITORING:
PASS / FAIL

LOGGING:
PASS / FAIL

BACKUPS:
PASS / FAIL / NOT IMPLEMENTED

RESTORE:
PASS / FAIL / NOT TESTED

DISASTER RECOVERY:
PASS / FAIL

CI/CD:
PASS / FAIL

ANDROID BUILD:
PASS / FAIL

DEPLOYMENT:
PASS / FAIL

ROLLBACK:
PASS / FAIL / NOT TESTED

JOBS/QUEUES:
PASS / FAIL / NOT IMPLEMENTED

COST CONTROL:
PASS / FAIL

OPERATIONS:
PASS / FAIL

SECURITY:
PASS / FAIL

TESTS:
PASS / FAIL

DOCUMENTATION:
PASS / FAIL

=====
ACTUAL TEST RESULTS
=====

Report actual results only.

Health:
[actual result]

API:
[actual result]

Monitoring:
[actual result]

Backup:
[actual result]

Restore:
[actual result]

CI:
[actual result]

Android build:
[actual result]

Deployment:
[actual result]

Rollback:
[actual result]

Jobs/queues:
[actual result]

Do NOT invent results.

=====
CRITICAL OPERATIONAL FINDINGS
=====

List:

CRITICAL
HIGH
MEDIUM
LOW

findings.

For each:
- component
- problem
- impact
- evidence
- remediation
- status

Never include secrets.

=====
BLOCKERS
=====

List every issue preventing:

PHASE 7 = READY

=====
NON-BLOCKING ISSUES
=====

List remaining improvements.

=====
STOP CONDITION
=====

Do NOT start Phase 8.

Do NOT add paid services without explicit approval.

Do NOT expose secrets.

Do NOT perform destructive production tests.

Do NOT fabricate uptime, backup, restore, monitoring or test results.

Do NOT delete legitimate cemetery or historical data.

STOP after the final report.

WAIT FOR EXPLICIT INSTRUCTIONS FOR PHASE 8.